

Audit of Black Sky Bound v2 Rendering Pipeline

Background

Developing a 2D game engine with the HTML `<canvas>` API often begins smoothly, but performance drops sharply once multiple layers, effects and post-processing are added. Browser dev-tools are the first place to confirm whether the drop-off is due to CPU-bound scripting (heavy loops, sorting, physics) or GPU-bound rendering (expensive pixel operations or too many draw calls) ¹. Key techniques to keep frame rates high on a canvas-driven game include:

- **Off-screen caching.** MDN's optimisation guide stresses that when a drawing operation is repeated in many frames, you should pre-render it to an off-screen canvas and simply stamp the cached image onto the main canvas ². This is especially useful for complex shapes or expensive operations such as text rendering. The AG-Grid team similarly notes that marking a scene "dirty" and redrawing only the changed layer into an off-screen canvas can **greatly enhance performance** ³.
- **Avoiding unnecessary state changes.** Changing a canvas' fill colour, line width, global alpha or transformation forces the browser to update its internal state machine. The web.dev article notes that state changes are expensive and that **batching draws by colour** (render all objects of one colour before switching) is faster than alternating colours ⁴.
- **Viewport (frustum) culling.** Objects outside the camera's view do not need to be rendered; skipping them reduces draw calls. The infinite-canvas performance lesson emphasises that graphics outside the viewport should be culled to reduce the number of draw calls ⁵. A simple axis-aligned bounding box per object can be used to test whether it intersects the viewport ⁶.
- **Batching operations and grouping paths.** The web.dev guide explains that drawing multiple primitives in one path (e.g., building a polyline instead of separate `lineTo` calls) reduces the overhead of repeated calls and improves performance ⁷.
- **Layering canvases.** When different parts of a scene have different update rates (e.g., static background versus animated foreground), rendering them on separate canvases allows the browser to composite layers via the GPU. MDN recommends using multiple layered canvases for complex scenes ⁸.

The following audit compares Black Sky Bound v2's current rendering architecture with these best practices and identifies improvements.

What the current engine already does

Canvas context. The game uses a single `<canvas>` with a 2D rendering context; there is no WebGL usage. It scales the canvas based on device-pixel-ratio and calls `ctx.scale(dpr, dpr)` once per frame to handle high-DPI displays. A fixed-timestep game loop decouples physics from rendering.

Off-screen canvas caching. Several render layers use hidden canvases to cache expensive drawing operations:

- **Decal layer** – stamp decals (blood, scorch marks) are drawn onto a hidden canvas once and reused until the decal map changes. When drawing, the cached image is blitted to the main canvas.
- **Lighting composite layer** – a separate off-screen canvas is used for the light buffer and another for the darkness overlay. Lights are drawn onto the light canvas, occlusion shadows are composited, and the darkness canvas holds the result. The main canvas then draws the darkness canvas in one call.
- **Smoke/atmosphere layer** – separate density, smoke, lighting and scatter textures are rendered into off-screen canvases at a reduced resolution before being warped and composited onto the main canvas. The game intentionally scales these buffers down (e.g., half-resolution) to reduce pixel operations.
- **Post-processing layer** – bloom and smoothing effects are run on a low-resolution off-screen canvas (controlled by a quality profile). Stable dithering patterns and blurred scenes are generated once per frame in off-screen buffers and then composited back.

This design follows MDN's advice to pre-render repeated or complex primitives off-screen ² and AG-Grid's recommendation to redraw only dirty layers ³. However, the current engine does not use the newer `OffscreenCanvas` API; instead, it creates hidden `<canvas>` elements on the main thread. While this still reduces draw calls, it keeps all work on the main thread.

Frustum and light-space culling. Each render layer converts world coordinates to screen coordinates and **skips drawing when the object's bounding box or circle is outside the viewport** (for tiles, actors, effects and napalm droplets). This is classic frustum culling for a 2D engine and follows the infinite-canvas recommendation that objects outside the viewport should not be rendered ⁵. In addition, the engine implements **light-space culling**: it merges the screen-space regions illuminated by active lights and skips detail rendering (terrain detail, actors, effects, napalm pools and smoke sources) outside these regions. This reduces work further when only part of the screen is illuminated.

Batching and grouping operations. Within each draw call the engine often sets up the drawing state (e.g., line colours, gradients) and draws the primitive immediately. There is no explicit batching of shapes with the same style; each actor or tile is drawn with its own `ctx.save()` / `ctx.restore()`. The smoke layer does group all lights together into a single off-screen layer and uses the browser's `destination-in` composite operation to mask them, which is a form of batching. The lighting composite also draws all lights in a loop before switching to bloom mode.

State changes. Because each actor or tile saves and restores the drawing context, many properties – fill colour, stroke colour, line width and global alpha – are set repeatedly. The number of actors and tiles is modest thanks to tight render budgets, but there is room to reduce repeated state changes by grouping similar objects. The engine avoids heavy native `ctx.filter` calls inside loops and performs custom post-processing via off-screen buffers.

Layered rendering. The renderer draws terrain, decals, actors, effects, lighting, smoke, post-processing and HUD in sequence on one canvas. Off-screen caches are used, but the final composition is done on a single canvas rather than separate layered canvases. For a simple top-down game this is adequate; layered canvases would help if the HUD or background needed to update at different rates.

What still needs improvement

Off-screen caching opportunities

- **Terrain caching.** Terrain tiles are static but are redrawn tile-by-tile on every frame. MDN suggests pre-rendering repeating primitives on an off-screen canvas ². A further optimisation would be to render the entire terrain map to an off-screen buffer once (or when the zoom level changes) and then blit the appropriate region of that buffer to the main canvas each frame. This avoids iterating through all tiles when the camera moves slowly.
- **Use of `OffscreenCanvas` API.** Modern browsers support `OffscreenCanvas`, which allows rendering to occur on a worker thread. Offloading expensive post-processing (blur, bloom, scatter) to a worker could free the main thread. However, adoption depends on browser support and complexity; initial results often show that `OffscreenCanvas` helps when heavy pixel manipulation is required ⁹.

Minimising state changes

The engine performs a `ctx.save()` / `ctx.restore()` for every actor, effect and tile, resulting in frequent state pushes. According to web.dev, changing the fill style or other state is expensive and it's cheaper to **render by colour** – draw all objects that share the same style consecutively ⁴. Potential improvements include:

- **Group drawing by style.** Sort actors or effects by their fill/stroke colour and draw them in batches, changing `ctx.fillStyle` as few times as possible. For example, draw all enemies with the same colour together before moving to the next colour.
- **Limit `save()` / `restore()` usage.** Many `save()` calls are only needed to alter `globalAlpha`. When possible, compute the alpha per actor and set `ctx.globalAlpha` directly without saving and restoring the full state.
- **Reuse gradients or patterns.** Creating new radial gradients inside loops causes object allocations. Cache gradients per light or per smoke profile outside the draw loop if the parameters do not change each frame.

Optimising culling and iteration

- **Spatial partitioning for actors and effects.** Currently the renderer loops through every actor and effect each frame and checks whether it intersects the viewport or light-space region. If the number of entities grows, consider organising entities into spatial grids or quad-trees so that only nearby items are tested against the view.
- **Reduce map iteration.** The terrain loop iterates over every tile in the map (not just those near the camera). For large maps, maintain indices of the visible tile range (e.g., compute start/end tile indices based on the camera and tile size) and iterate only that range. This is consistent with viewport culling advice ⁶.

Other suggestions

- **Batch drawing commands.** Where appropriate, combine multiple primitives into one path. For example, when drawing grid lines or health bars, build a single path and stroke it once ⁷.
- **Consider WebGL for further scaling.** If the game's scope grows and the number of entities or visual effects increases, migrating to a WebGL-based renderer may provide better performance.

WebGL allows sprite batching and texture atlasing, reducing draw calls drastically ³. However, it also requires more complex management and shader programming.

- **Multiple layered canvases.** Splitting the HUD, terrain, and effects into separate `<canvas>` elements would allow the browser to composite layers efficiently, especially when the HUD or static background rarely changes ⁸. This technique can be applied incrementally.

Plan of action

1. **Measure current bottlenecks.** Before changing code, profile the game using the browser's performance panel to see whether the CPU time is dominated by scripting or by drawing. Enable FPS and timeline markers around tile drawing, actor drawing, lighting and post-processing to identify hotspots.
2. **Introduce terrain caching.** Create an off-screen canvas the size of the world map (or a region sized to the viewport at current zoom). Render the terrain tiles to this buffer when the map or zoom level changes. In each frame, draw only the visible portion of this buffer onto the main canvas, adjusting for camera position. This should reduce per-tile drawing and repeated state changes for terrain.
3. **Group drawing and reduce state changes.** In actor and effect layers, group entities by style. For example, separate arrays for different enemy colours and iterate each array in sequence, setting `fillStyle` once. Avoid saving/restoring state for simple alpha changes; compute `ctx.globalAlpha = baseAlpha * lightAlpha` directly. Cache radial gradients for lights and smoke to avoid creating new gradient objects each frame.
4. **Implement tile and entity range culling.** Compute the range of tile indices visible in the camera's viewport and iterate only those tiles when drawing terrain. For actors, organise entities into spatial buckets (rows or a grid) and iterate only buckets intersecting the viewport or light-space.
5. **Evaluate use of `OffscreenCanvas`.** Experiment with moving the post-processing pipeline (bloom and smoothing) to an `OffscreenCanvas` running in a worker. Measure whether offloading pixel operations increases frame rates on target browsers.
6. **Optional long-term improvements.** If future slices add more enemies or effects, consider porting the renderer to WebGL for built-in sprite batching and shader-based post-processing. Also consider splitting the game into multiple `<canvas>` layers to isolate the HUD and static backgrounds.

By comparing the current implementation against widely-recommended optimisation techniques ² ⁴ ⁵ ³, this audit highlights areas where Black Sky Bound v2 already follows best practices (layer caching and culling) and areas where further optimisation could help maintain high frame rates as the game scales.

¹ Optimizing canvas - Web APIs | MDN

https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API/Tutorial/Optimizing_canvas

2 4 7 8 9 Improving HTML5 Canvas performance | Articles | web.dev
<https://web.dev/articles/canvas-performance>

3 Optimising HTML5 Canvas Rendering: Best Practices and Techniques
<https://blog.ag-grid.com/optimising-html5-canvas-rendering-best-practices-and-techniques/>

5 6 Lesson 8 - Optimize performance | An infinite canvas tutorial
<https://antv.vision/infinite-canvas-tutorial/guide/lesson-008>